OWASP-ZAP

Prepared By:

Hanamariam Yehuala

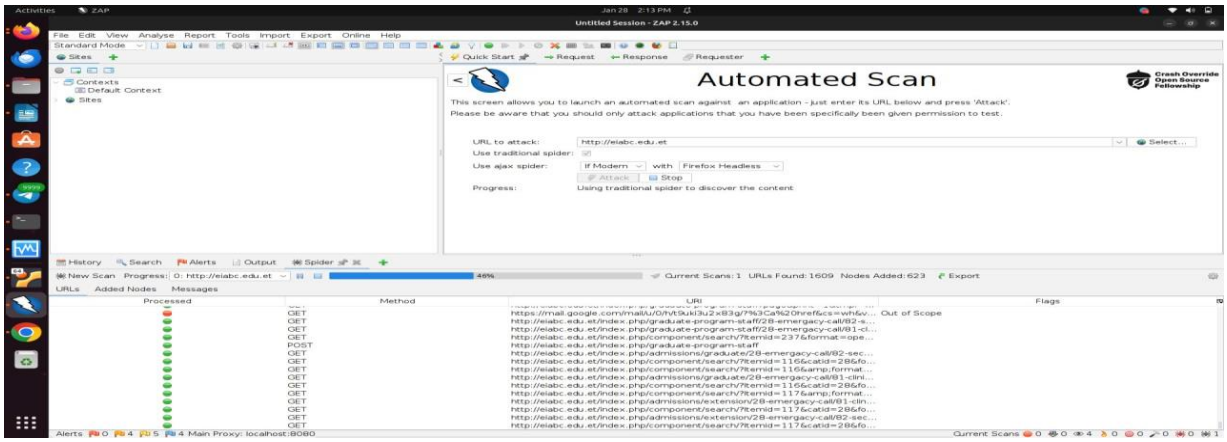
Security Assessment Report for EiABC Website

1. Introduction

- **Purpose**: This report details the security assessment of the web application http://eiabc.edu.et using the OWASP Zed Attack Proxy (ZAP). The goal was to identify potential vulnerabilities and evaluate the application's resilience to common web application security risks.
- **Scope**: The scope of this assessment includes:
  - Scanning the web application for vulnerabilities.
  - Identifying weaknesses that could be exploited by attackers.
  - Providing recommendations for improving the overall security posture.

Methodology

1. **Tool Used**: OWASP ZAP, an open-source web application security scanner.
2. **Assessment Process**:
   - **Active Scanning**: Automated scans to identify vulnerabilities.
   - **Reporting**: Compiling findings and recommendations into a structured report.



Findings

1. Absence of Anti-CSRF Tokens

- **Vulnerability**: Forms on the website lack CSRF protection mechanisms.
- **Impact**: Without CSRF tokens, attackers can perform unauthorized actions on behalf of users.
- **Recommendation**: Implement CSRF tokens in all forms to mitigate this vulnerability.

2. Application Error Disclosure

- **Vulnerability**: The application may disclose detailed error messages.
- **Impact**: Attackers can gain insights into the application's architecture and potential weaknesses.
- **Recommendation**: Use generic error messages and avoid exposing stack traces or sensitive information.

3. Content Security Policy (CSP) Header Missing

- **Vulnerability**: The website does not implement a Content Security Policy.
- **Impact**: This absence increases the risk of XSS and data injection attacks.
- **Recommendation**: Configure a CSP header to specify allowed sources for scripts and other content.

4. Missing Anti-Clickjacking Protection

- **Vulnerability**: The application lacks protections against clickjacking attacks.
- **Impact**: Users may be tricked into clicking on something different from what they perceive, potentially leading to unauthorized actions.
- **Recommendation**: Implement the `X-Frame-Options` header to prevent the site from being framed.

5. Missing X-Content-Type-Options Header

- **Vulnerability**: The website does not include the `X-Content-Type-Options` header.
- **Impact**: This omission allows browsers to perform MIME type sniffing, which can lead to security risks.
- **Recommendation**: Set the `X-Content-Type-Options` header to `nosniff` to prevent MIME type sniffing.

6. Vulnerable JavaScript Libraries

- **Vulnerability**: Identified vulnerable JavaScript libraries in use.

- **Impact**: Known vulnerabilities in these libraries can be exploited by attackers.
- **Recommendation**: Regularly audit and update JavaScript libraries to mitigate vulnerabilities.

7. Cookie Without Secure and HttpOnly Flags

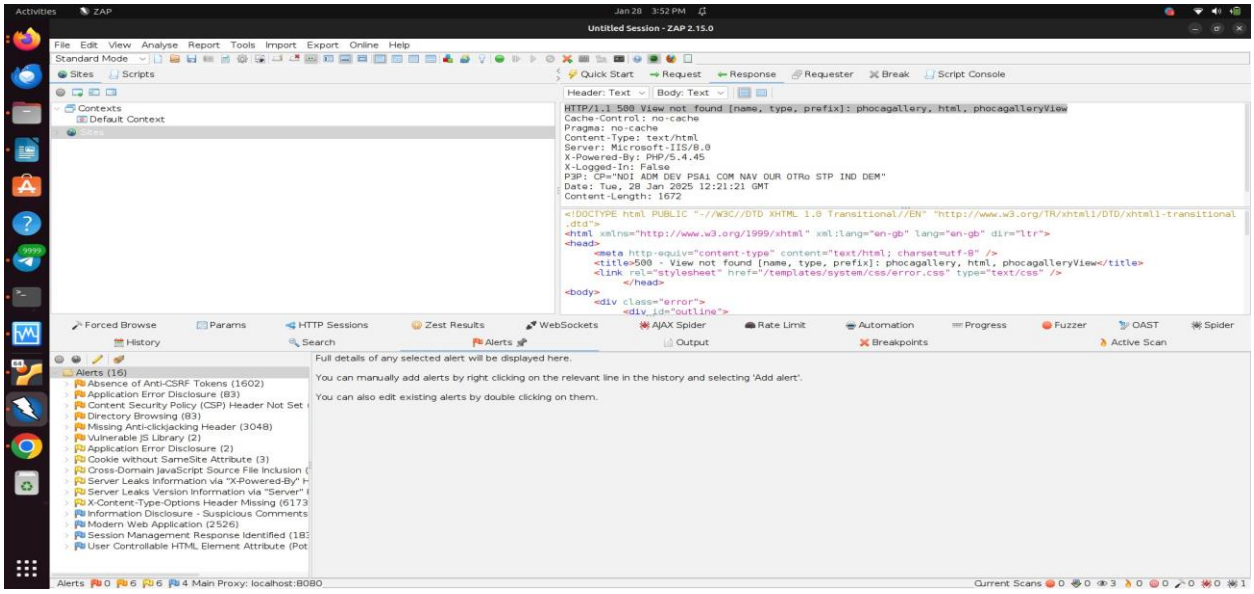
- **Vulnerability**: Cookies are not set with the `Secure` and `HttpOnly` flags.
- **Impact**: This increases the risk of session hijacking and cross-site scripting attacks.
- **Recommendation**: Ensure cookies have the `Secure` and `HttpOnly` flags set to enhance security.

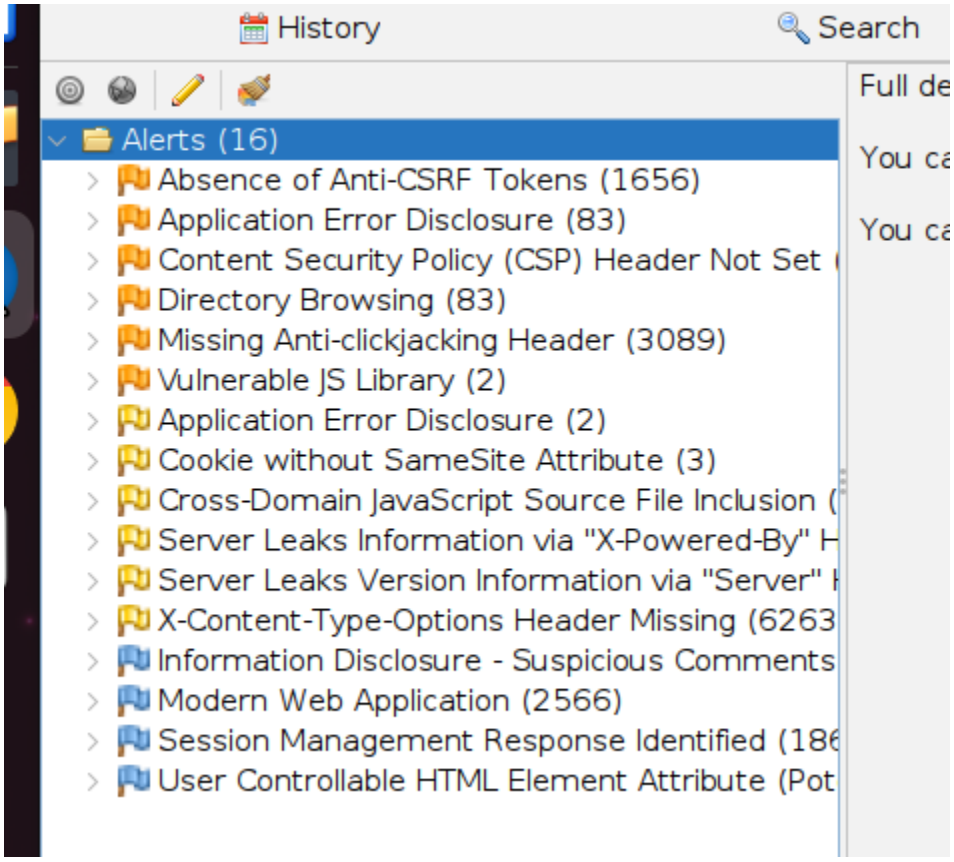
8. Information Disclosure - Suspicious Comments

- **Vulnerability**: The presence of comments in the code may disclose sensitive implementation details.
- **Impact**: Attackers can use this information to exploit the application.
- **Recommendation**: Remove any comments that contain sensitive information or implementation details before deployment.

9. User Controllable HTML Element Attribute

- **Vulnerability**: The application allows users to control HTML attributes.
- **Impact**: This can lead to XSS vulnerabilities if not properly sanitized.
- **Recommendation**: Ensure that user inputs are adequately validated and sanitized to prevent injection attacks.





Recommendations

1. **Implement CSRF Protection**: Add anti-CSRF tokens to all forms.
2. **Enhance Error Handling**: Use generic error messages to prevent information leakage.
3. **Configure CSP**: Implement a Content Security Policy to mitigate XSS risks.
4. **Add Clickjacking Protection**: Set the `X-Frame-Options` header.
5. **Set Security Headers**: Include `X-Content-Type-Options: nosniff`.
6. **Audit JavaScript Libraries**: Regularly update libraries to fix vulnerabilities.
7. **Secure Cookies**: Ensure cookies are set with `Secure` and `HttpOnly` flags.
8. **Remove Sensitive Comments**: Clean up code comments that may disclose sensitive information.
9. **Sanitize User Inputs**: Validate and sanitize all user-controllable inputs to prevent XSS.

Conclusion

The assessment revealed several critical vulnerabilities that could compromise the security of the EiABC website. By implementing the recommended security measures, the organization can

significantly improve its security posture and protect its users from potential attacks. Regular security assessments and updates should be part of the organization's ongoing security strategy.

Appendix

- **Tools Used**: OWASP ZAP
- **Assessment Date**: Jan 28, 2025